

Universidade Presbiteriana Mackenzie
Mestrado Profissional em Computação Aplicada

Professora: Valéria Farinazzo Martins

Disciplina: Algoritmos e Programação

Aluno: Cândido Francisco Moreira

RA: 10779363

Período: 1º Semestre de 2026

Projeto 2

Comparação entre árvores

São Paulo, 07 de abril de 2026

1. Metodologia

O presente tópico descreve a metodologia experimental, o ambiente computacional e as decisões técnicas adotadas para a execução dos testes de *benchmarking* comparativos entre as estruturas de Árvore Binária de Busca (BST) e Árvore AVL.

1. Ambiente Computacional e Hardware

Para garantir a reprodutibilidade dos resultados e mitigar interferências do sistema operacional, os experimentos foram conduzidos em um ambiente com isolamento de processos e configuração de afinidade de CPU.

- **Sistema Operacional:** Windows 11 (Build 10.0.26200)
- **Processador:** Intel64 Family 6 Model 183 Stepping 1 (20 núcleos físicos / 28 *threads* lógicas)
- **Memória RAM:** 64 GB
- **Isolamento e Prioridade:** Os testes foram executados com prioridade alta (*High Priority*) e travas de isolamento de núcleo, reduzindo a variação de escalonamento do sistema operacional e ruídos experimentais.

2. Linguagem e Configurações de Execução

O projeto foi implementado em **Python 3.12.4**. Para capturar o desempenho real das estruturas de dados e evitar interferências não determinísticas na medição de tempo:

- O coletor de lixo (*Garbage Collector*) foi temporariamente desativado durante os blocos críticos de inserção e busca.
- A cronometragem foi realizada utilizando o relógio de alta resolução da biblioteca padrão (`time.perf_counter()`).

3. Base de Dados e Massa de Testes

A massa de dados foi extraída do arquivo `wb_commodity_price_intelligence_1960_2026.csv`. Os dados utilizados foram obtidos a partir de um dataset disponibilizado na plataforma Kaggle, baseado na base de preços de commodities do Banco Mundial. Cada registro foi mapeado para os nós das árvores utilizando uma chave unívoca composta pela concatenação das colunas `commodity_code` e `date`.

Para avaliar a escalabilidade das estruturas, o tamanho da entrada (N) foi fracionado em cinco volumes distintos:

- **10.000** registros
- **20.000** registros
- **30.000** registros
- **40.000** registros
- **49.093** registros (dataset completo)

4. Cenários de Ordenação

Como a disposição inicial dos dados afeta drasticamente a topologia das árvores de busca, os testes de inserção e busca foram submetidos a três cenários prévios de ordenação:

- **Crescente (*Ordered*)**: Inserção na ordem cronológica/alfabética original
- **Decrescente (*Reversed*)**: Inserção em ordem inversamente proporcional, para avaliar o comportamento do pior caso no sentido oposto.
- **Aleatório (*Shuffled*)**: Dados previamente embaralhados, simulando um cenário de distribuição ideal de entropia para a criação de árvores balanceadas naturalmente.

5. Estruturas e Algoritmos Avaliados

Duas estruturas de dados fundamentais foram implementadas e comparadas:

- **Árvore Binária de Busca (BST)**: Implementação padrão sem balanceamento. Possui complexidade de tempo de busca de $O(\log n)$ no caso médio, mas degrada para $O(n)$ no pior caso (dados já ordenados).
- **Árvore AVL**: Estrutura auto-balanceada que aplica rotações simples e duplas (esquerda/direita) durante a inserção para garantir que o fator de balanceamento dos nós permaneça entre -1 e 1. Garante complexidade rigorosa de $O(\log n)$ em todos os casos.
-

6. Estratégia de Execução e Paralelismo

As rotinas de inserção e busca foram submetidas a duas abordagens de execução:

- **Execução Sequencial (*Single-Thread*)**: Utilizada como *baseline*, processando as requisições em um único núcleo lógico.
- **Execução Multi-tarefas (*Multiprocessing* em 4 Cores)**: Distribuição da carga entre 4 processos isolados utilizando a biblioteca `multiprocessing`. Esta abordagem foi introduzida especificamente para avaliar e quantificar o impacto do *Global Interpreter Lock* (GIL) do Python, bem como os custos associados de serialização (*Pickle*) e do atraso de Comunicação Inter-Processos (IPC Delay) em operações intensivas de memória.

7. Métricas Coletadas

Para a fundamentação da análise crítica, as seguintes métricas foram registradas:

- Tempo absoluto de inserção (em segundos/milissegundos).
- Tempo absoluto de busca por chaves específicas.
- Altura final (níveis) gerada para cada árvore nos diferentes cenários.

2. Gráficos

A seguir apresentamos as visualizações gráficas obtidas na análise de desempenho, comparando execuções sequenciais (*Single-Thread*) e paralelas (*Multiprocessing* em 4 núcleos). O foco desta etapa é quantificar o impacto da paralelização sobre o tempo de busca nas estruturas de Árvore Binária de Busca (BST) e Árvore AVL.

Os experimentos foram conduzidos com diferentes volumes de dados (de 10.000 a 49.093 registros) e sob três cenários de ordenação da massa de dados (Crescente, Decrescente e Aleatório), com o objetivo de observar o comportamento assintótico real das estruturas em condições de estresse distintas.

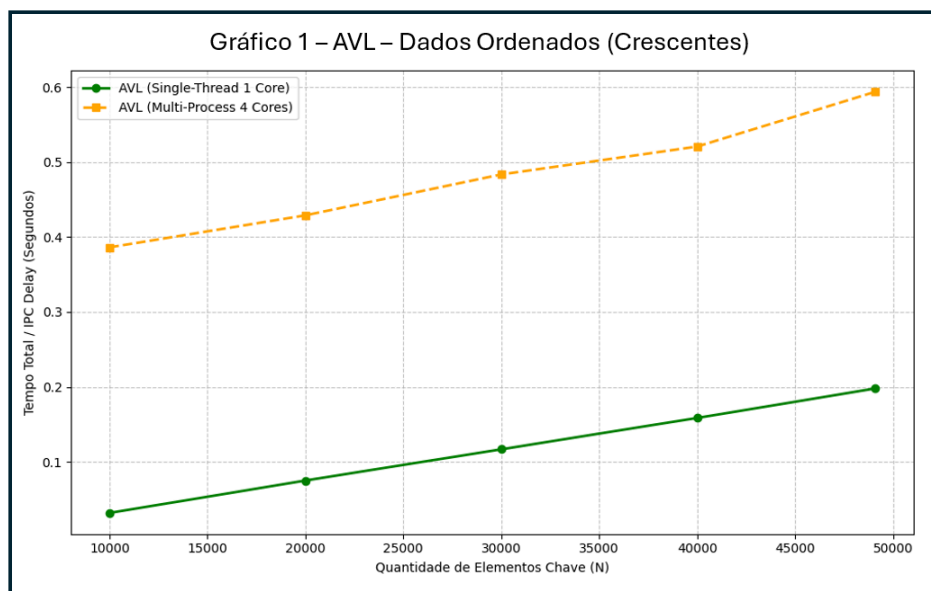
1. Árvore AVL – O Paradoxo do IPC Delay

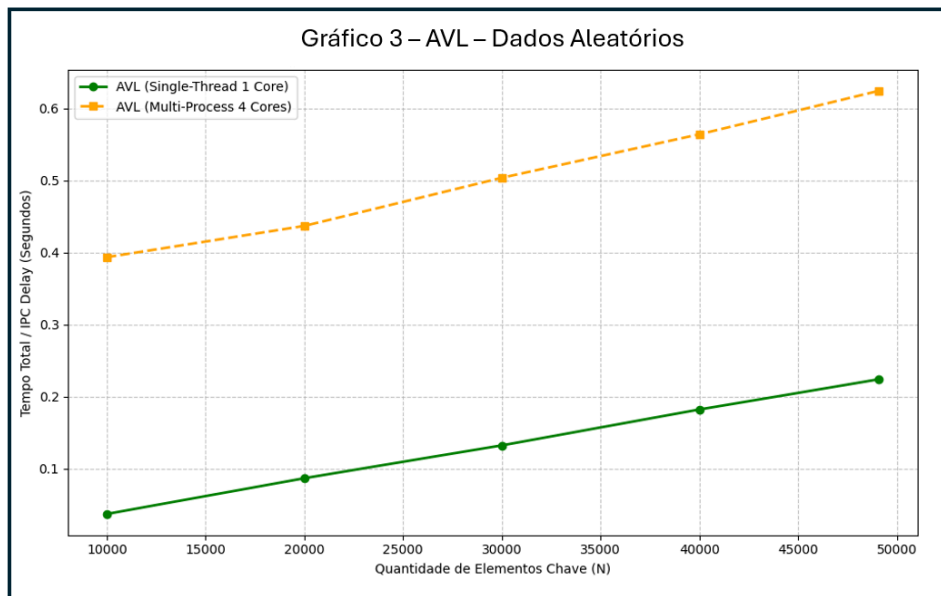
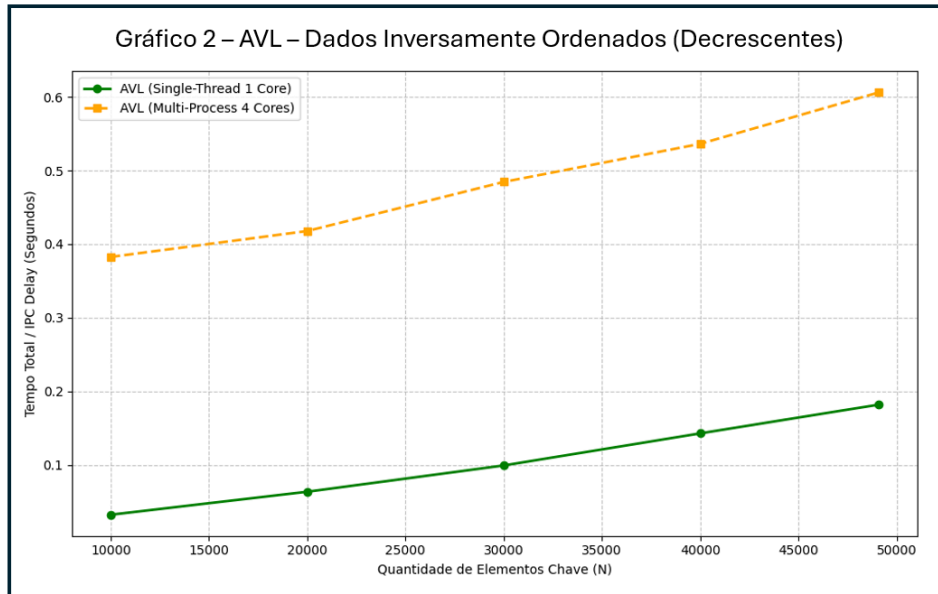
A Árvore AVL, por ser uma estrutura estritamente auto-balanceada, mantém sua altura sempre limitada a $O(\log N)$, garantindo uma eficiência de busca consistente e extremamente rápida de forma nativa.

Os resultados gráficos demonstram que, para esta estrutura, a utilização de *multiprocessing* não trouxe ganhos de desempenho práticos. Pelo contrário, observou-se um aumento astronômico no tempo total de execução. Este comportamento paradoxal é explicado pelo elevado custo arquitetural da paralelização em Python:

- **Serialização de Dados (*Pickling*):** A necessidade de converter os nós da árvore em fluxos de bytes para tráfego entre processos.
- **Comunicação Inter-Processos (*IPC Delay*):** O tempo gasto para transferir a massa de dados entre as memórias isoladas dos diferentes trabalhadores (*workers*).
- **Sobrecarga de Gerenciamento:** O custo do sistema operacional para orquestrar os processos superou amplamente os milissegundos economizados na busca.

(Como as operações na AVL já são eficientes na execução sequencial, o custo do paralelismo anula qualquer possível benefício computacional).





2. Árvore BST – Degradação Estrutural vs. Paralelismo

Diferentemente da AVL, a Árvore Binária de Busca (BST) não possui mecanismo de balanceamento automático. Consequentemente, sua topologia e performance dependem exclusivamente da ordem de inserção dos dados, o que altera drasticamente o impacto do paralelismo.

2.1 Cenários Ordenados (Crescente e Decrescente)

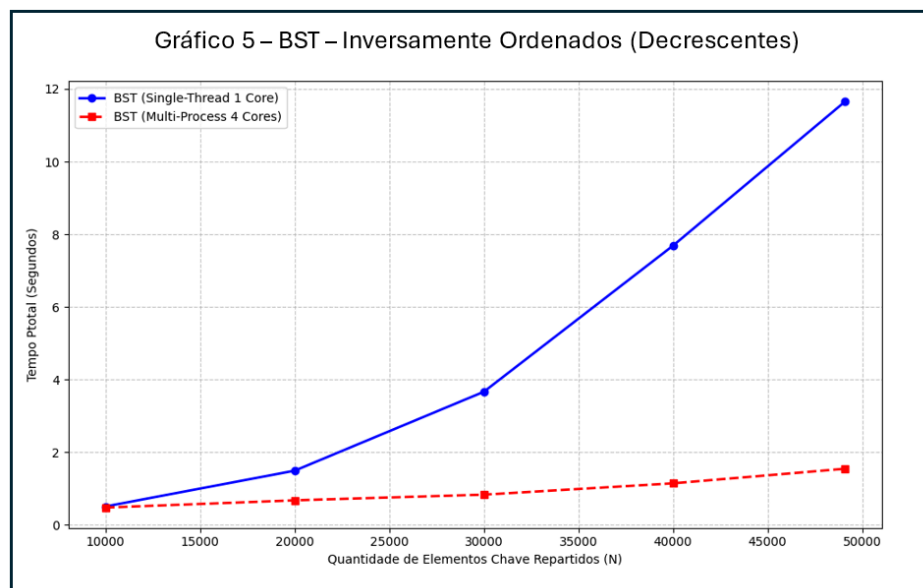
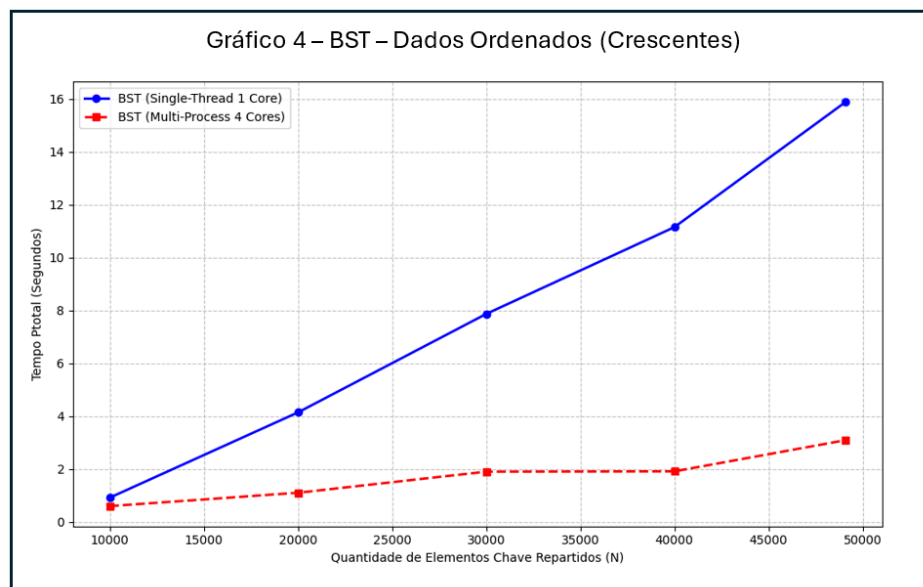
Quando alimentada com dados já ordenados, a BST sofre uma degeneração estrutural severa, assumindo o formato de uma lista encadeada diagonal. Nesses casos, a altura da árvore aproxima-se de $O(N)$ e o tempo de busca degrada de forma linear.

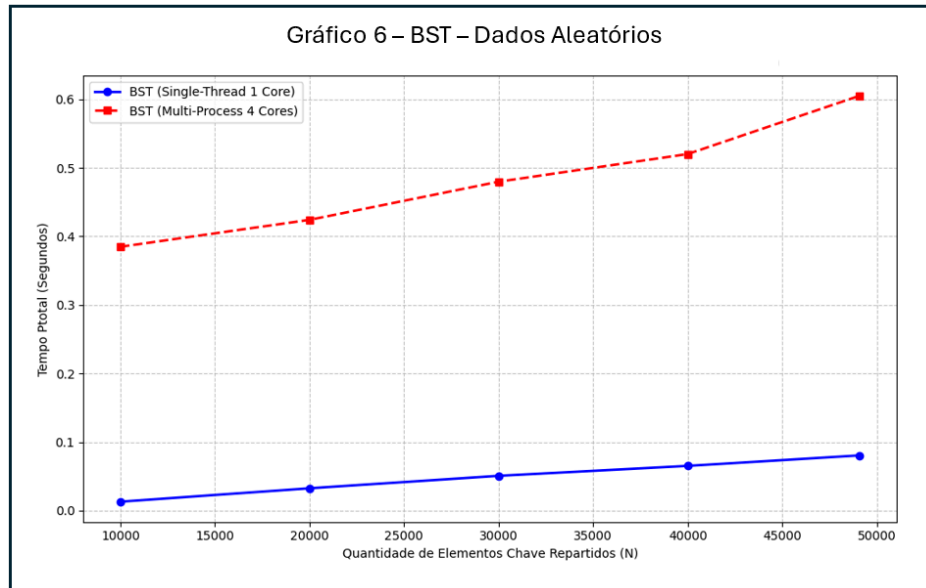
Sob este estresse drástico na execução *Single-Thread* (onde o algoritmo consome muitos segundos iterando sequencialmente), o acionamento de múltiplos núcleos demonstra certa eficácia: a distribuição bruta da carga de trabalho entre os processos consegue mitigar parte da lentidão estrutural da árvore, atenuando as perdas mesmo com o *overhead* do IPC.

2.2 Cenário Aleatório (*Shuffled*)

Quando os mesmos dados são inseridos de forma aleatória, a BST tende a formar uma estrutura naturalmente distribuída, com altura próxima a $O(\log N)$.

Neste cenário de entropia ideal, a árvore volta a apresentar alta agilidade em execução *Single-Thread*. Consequentemente, ao forçarmos o *Multiprocessing*, a necessidade de paralelização torna-se nula e os enormes custos de comunicação inter-processos (*Pickle/IPC*) voltam a penalizar severamente os tempos de execução, assim como ocorreu na AVL.





3. Síntese Comparativa

A análise das métricas gráficas permite consolidar as seguintes premissas:

- A Árvore AVL apresenta desempenho estável, ágil e imune à ordem de entrada dos dados.
- A Árvore BST é altamente volátil, dependendo da entropia (aleatoriedade) da massa de dados para manter sua eficiência.
- O paralelismo em Python (via *multiprocessing*) não é vantajoso para operações de busca em estruturas já eficientes (AVL ou BST balanceada). O ganho da computação paralela nesta arquitetura só se justifica em cenários de degradação extrema (como a BST degenerada em lista).

3. Análise Crítica sobre a Eficiência dos Algoritmos para Inserção e para Busca.

Com base nas métricas obtidas no ambiente de experimentação controlado (com afinidade de CPU, isolamento de processos e desativação do *garbage collector*), as análises de eficiência revelaram diferenças comportamentais significativas entre as estruturas BST e AVL. A variação no volume de dados (até aproximadamente 49.000 registros) e nos cenários de ordenação permitiu isolar o impacto da arquitetura de cada árvore frente ao uso de processamento sequencial e paralelo.

1. Árvore Binária de Busca (BST) – Degradação Estrutural e Mitigação via Paralelismo

Nos cenários de inserção com dados previamente ordenados (crescente e decrescente), a BST apresentou forte degeneração estrutural. Sua altura aproximou-se de $O(N)$, assumindo o comportamento prático de uma lista encadeada, o que tornou a complexidade das operações linear.

- **Execução Sequencial (*Single-Thread*):** Para a carga limítrofe de ~49k elementos, a ineficiência algorítmica foi severamente penalizada, exigindo **15,8s** no cenário crescente e **11,6s** no decrescente.
- **Execução Paralela (*Multiprocessing com 4 Cores*):** Ao distribuir essa mesma carga degradada entre múltiplos trabalhadores, o tempo total foi reduzido para **~3,09s** (uma aceleração de aproximadamente 80%). Este resultado comprova que, em situações de degradação estrutural extrema e alta latência iterativa, escalar horizontalmente a carga de trabalho não corrige a degradação estrutural, mas reduz o tempo total de execução ao distribuir a carga entre múltiplos processos.

2. Árvore AVL – Robustez Algorítmica e o Gargalo do Paralelismo

A Árvore AVL apresentou um comportamento consistente e previsível em todos os cenários, mantendo sua altura estritamente balanceada em $O(\log N)$ através de rotações de compensação, independentemente da ordem de inserção dos dados.

- **Execução Sequencial:** A estrutura demonstrou altíssima eficiência, alocando a massa completa de dados em tempos que variaram apenas entre **0,19s e 0,22s**.
- **Execução Paralela:** Ao ativar o *multiprocessing*, o tempo de execução sofreu um aumento drástico (chegando a **+1100%** de lentidão). Como a AVL já é nativamente rápida, a imposição de múltiplos núcleos criou um gargalo. O pesado custo de serialização de objetos (*Pickling*) e o atraso na comunicação entre processos (IPC) superaram largamente qualquer ganho computacional.

3. Eficiência sob Entropia (O Cenário Aleatório)

No cenário com dados previamente embaralhados (*Shuffled*), a dinâmica de eficiência sofreu uma inversão notável. A injeção de dados com alta entropia permitiu que a BST formasse uma topologia naturalmente bem distribuída, com altura média próxima a $O(\log N)$.

- Neste cenário ótimo, a BST finalizou a inserção dos 49k registros em expressivos **0,08s** (em *Single-Thread*), superando o desempenho da própria AVL (que se manteve em ~0,22s).
- Essa vantagem prática da BST ocorre porque ela não possui o custo computacional (*overhead*) de verificação de fatores de balanceamento e execução de operações de rotação a cada inserção, algo obrigatório na AVL.
- Tentar aplicar paralelismo a este cenário ideal resultou em perdas severas para ambas as árvores (de **+600% a +900%** de lentidão frente à execução sequencial bruta).

4. Síntese da Análise

A correlação entre as estruturas de dados e a arquitetura da linguagem Python permite concluir que:

- **A topologia dita a performance:** A BST é altamente vulnerável à ordem de entrada dos dados, enquanto a AVL oferece garantias de tempo à custa de processamento contínuo de balanceamento.
- **Simplicidade vence em cenários ideais:** Quando a distribuição dos dados garante uma árvore balanceada naturalmente (cenário aleatório), a ausência de algoritmos de rotação torna a BST mais rápida que a AVL.
- **O limite do paralelismo em Python:** A paralelização de processos (*multiprocessing*) não substitui a eficiência algorítmica. Para estruturas rápidas e de baixa complexidade $O(\log N)$, os custos arquiteturais de paralelização em Python — notadamente a serialização de objetos (*Pickle*) e a comunicação inter-processos (IPC) — inviabilizam o ganho de desempenho. A aplicação de *multiprocessing* mostrou-se útil apenas para atenuar o impacto temporal no pior caso teórico da BST $O(N)$, reduzindo o tempo percebido sem, contudo, alterar a ineficiência intrínseca da estrutura gerada.

4. Análise Crítica sobre a Análise Assintótica

A análise assintótica clássica (notação Big-O) foi confrontada com os resultados empíricos obtidos no ambiente de execução, permitindo avaliar não apenas o limite teórico de crescimento das estruturas de dados, mas também os impactos práticos de fatores arquiteturais, como o *overhead* de comunicação inter-processos (IPC). Os experimentos, conduzidos com volumes crescentes de dados (de $N = 10.000$ até $N = 49.093$), evidenciaram como os custos computacionais ocultos podem mascarar a eficiência assintótica.

1. Árvore AVL – Estabilidade Assintótica e o Domínio do Custo Linear

Do ponto de vista teórico, a Árvore AVL garante uma complexidade estrita de $O(\log N)$ para operações de busca e inserção no pior caso. Os resultados empíricos em execução sequencial (*Single-Thread*) confirmaram esse comportamento, apresentando um crescimento de tempo incrivelmente suave e previsível, imune à ordem de inserção dos dados.

Entretanto, ao introduzir o paralelismo via *multiprocessing*, observou-se uma degradação severa que chegou a +1116% de tempo envolvido. É fundamental ressaltar que a complexidade assintótica algorítmica da AVL não sofreu alteração; o que mudou foi o custo constante associado à infraestrutura de execução. Na prática paralela, o tempo total percebido pode ser modelado como:

$$T(N) \approx \text{Tempo_Serializacao}(N) + \text{Tempo_Computacao}(\log N)$$

Onde o termo de serialização (processo de *Pickling* nativo do Python) cresce de forma linear $O(N)$ conforme a massa de dados aumenta. Consequentemente, esse custo infraestrutural linear passa a dominar a equação, ocultando o comportamento logarítmico $O(\log N)$ ideal da árvore e gerando a latência massiva observada.

2. Árvore BST – Variação Assintótica e os Limites do Paralelismo

A Árvore Binária de Busca (BST) apresenta um comportamento assintótico altamente dependente de sua topologia, flutuando entre $O(\log N)$ no caso médio e $O(N)$ no pior caso.

2.1 O Pior Caso (Cenários Ordenados) Nos cenários crescente e decrescente, a árvore degenera em uma estrutura linear (lista encadeada). Com a altura da árvore alcançando $O(N)$, cada nova inserção exige uma varredura $O(N)$, levando a complexidade total de construção da árvore para $O(N^2)$. Neste cenário de "Worst-Case" extremo, fatiar e despachar a carga horizontalmente para 4 núcleos reduziu ativamente a lentidão, com ganhos práticos entre 80% e 86% a favor do processo paralelo. Contudo, do ponto de vista da análise assintótica, a complexidade teórica permaneceu $O(N^2)$. O paralelismo atuou apenas reduzindo a constante de tempo da operação, mitigando as limitações do processamento sequencial sem alterar o crescimento quadrático do algoritmo.

2.2 O Caso Médio (Cenário Aleatório) No cenário com dados embaralhados, a entropia natural permite que a BST forme sub-árvores estatisticamente balanceadas, retomando a eficiência teórica de $O(\log N)$ na altura. Assim como ocorreu na AVL, ao forçar a paralelização sobre uma estrutura que já opera de forma rápida e eficiente, o custo infraestrutural volta a dominar. Sem a penalidade de uma varredura $O(N^2)$ para justificar a força bruta do multiprocessamento, a imposição de transferências IPC regrediu o desempenho severamente, gerando atrasos superiores a 2800% mesmo para volumes menores ($N = 10.000$).

3. Interpretação sob a Lei de Amdahl

Os fenômenos observados alinham-se perfeitamente aos postulados da Lei de Amdahl, que estabelece limites matemáticos para o ganho de desempenho via paralelização. No contexto deste estudo:

- A fração do problema que realmente se beneficia do paralelismo é limitada.
- Há custos adicionais pesados de coordenação, comunicação (IPC) e serialização (*Pickle*).
- Consequentemente, em cenários assintoticamente eficientes (operações nativas em $O(\log N)$), o ganho paralelo é nulo ou negativo. O paralelismo só apresentou utilidade prática no cenário de degradação máxima ($O(N^2)$), onde o ganho na fração paralelizável conseguiu superar o peso do *overhead* de comunicação.

4. Síntese Crítica

A análise conjunta dos dados evidencia que:

- A notação assintótica permanece válida como modelo teórico, mas fatores de implementação (como o GIL, *Pickling* e IPC do Python) introduzem constantes lineares que podem dominar o tempo real de execução.
- O paralelismo reduz o tempo absoluto em cenários de degradação estrutural pesada, mas não altera a ordem de complexidade matemática do algoritmo.
- Para operações em estruturas otimizadas (limite de $O(\log N)$), o ganho marginal de divisão de tarefas é esmagado pelo custo computacional do tráfego inter-processos, tornando a computação sequencial (*Single-Thread*) a abordagem mais eficiente.

5. Análise Crítica sobre a Análise Assintótica

Uma propriedade fundamental das Árvores Binárias de Busca — abrangendo tanto a topologia BST padrão quanto a estrutura auto-balanceada AVL — é que sua organização espacial e matemática depende exclusivamente de uma chave de ordenação primária definida no momento da inserção.

No contexto deste projeto, os nós foram indexados a partir de uma chave composta unívoca (`commodity_code + date`). Consequentemente, todas as garantias de eficiência assintótica direcional ($O(\log N)$ no caso balanceado) aplicam-se estrita e unicamente às buscas realizadas com base nesta chave primária.

1. O Impacto Estrutural da Busca por Campos Secundários

Quando a requisição de busca exige a filtragem por um atributo diferente (por exemplo, localizar todos os registros onde `commodity_name == "Aluminum"`), a estrutura da árvore perde completamente sua capacidade de direcionamento lógico (decisões de ir para o sub-galho à esquerda ou à direita).

Nesse cenário, a árvore deixa de oferecer vantagem algorítmica em relação a uma matriz não ordenada. O algoritmo é forçado a realizar uma travessia completa por todos os nós (*Tree Traversal* iterativo ou recursivo). Isso resulta em:

- **Complexidade de Tempo:** Degradação imediata para $O(N)$, pois há a necessidade de visitar e comparar potencialmente todos os nós da estrutura em busca de colisões favoráveis.
- **Nulidade do Balanceamento:** Independentemente de a árvore ser uma AVL perfeitamente balanceada ou uma BST degenerada, o tempo de busca por um campo não-chave será rigorosamente o mesmo, anulando os benefícios da estrutura.

2. Soluções e Alternativas Sistêmicas

Para contornar essa limitação arquitetural e suportar buscas por múltiplos campos, a literatura de estruturas de dados e bancos de dados propõe três abordagens principais, cada qual com seus próprios *trade-offs* de memória e processamento:

2.1 Varredura Completa (Escaneamento Iterativo $O(N)$) Consiste em aceitar a limitação e percorrer a árvore inteira nó a nó sempre que a busca secundária for requisitada.

- **Vantagem:** Simplicidade de implementação e custo zero de memória adicional (Complexidade de Espaço $O(1)$).
- **Desvantagem:** Altíssimo custo computacional ($O(N)$).
- **Aplicação:** Estratégia viável apenas se houver garantia empírica de que as consultas por campos secundários serão extremamente raras ou esporádicas.

2.2 Reindexação e Duplicação Estrutural Consiste em instanciar e manter na memória RAM uma segunda árvore binária completa, cujos nós contenham os mesmos dados, mas que utilize o campo secundário (`commodity_name`) como chave de ordenação primária.

- **Vantagem:** Recupera a eficiência de busca rigorosa de $O(\log N)$ para o novo critério.
- **Desvantagem:** Consumo redundante de memória (Complexidade de Espaço $O(N)$ adicional) e aumento do *overhead* de inserção, pois cada novo registro do dataset precisará ser inserido e balanceado em duas árvores distintas simultaneamente.

2.3 Índices Secundários via Hash Maps (Abordagem Recomendada) A solução mais elegante e amplamente adotada em sistemas de banco de dados modernos consiste em manter estruturas auxiliares de indexação associadas à árvore principal. Em Python, isso é feito emparelhando a árvore a um dicionário (*Hash Map*). Neste modelo, o dicionário utiliza o campo secundário (ex: "`Aluminum`") como chave, e armazena como valor um ponteiro (referência de memória) apontando diretamente para o nó correspondente dentro da árvore binária.

- **Vantagem:** Permite localizar o nó com complexidade de tempo média de $O(1)$, burlando os custosos *traversals* $O(N)$ sem a necessidade de duplicar a árvore inteira.
- **Desvantagem:** Requer manutenção paralela do dicionário durante as deleções e inserções.

3. Síntese Crítica

A análise das topologias binárias demonstra que elas são estruturas unidimensionais, altamente otimizadas para um único caminho de indexação. Consultas por atributos secundários eliminam as vantagens da árvore, exigindo soluções de engenharia adicionais. A escolha da estratégia de mitigação deve ser pautada pelo padrão de acesso aos dados: consultas esporádicas justificam a varredura linear $O(N)$, enquanto consultas frequentes em sistemas de produção exigem a implementação de Índices Secundários anexos à estrutura principal.

6. Análise Crítica sobre a Análise Assintótica

Para viabilizar a busca por um atributo não indexado pela chave primária da árvore (como o campo `commodity_name`), foi implementada uma rotina de varredura completa (*Tree Traversal*). A abordagem adotada utiliza uma estratégia iterativa baseada em pilha (*Stack*), realizando uma busca em profundidade (DFS - *Depth-First Search*). Esta escolha arquitetural evita o uso de recursão, prevenindo possíveis estouros de limite de chamadas (*RecursionError*) do interpretador Python em casos de árvores altamente degeneradas.

1. Código Implementado

A rotina abaixo ilustra a varredura iterativa, que empilha sistematicamente cada nó da estrutura para validar o conteúdo de seu registro:

Python

```
def search_by_attribute(self, attribute_name, value):
    results = []
    if not self.root:
        return results

    # Utilização de pilha (stack) para travessia iterativa (DFS)
    stack = [self.root]

    while stack:
        node = stack.pop()

        # Validação do atributo no registro associado ao nó
        if node.record.data.get(attribute_name) == value:
            results.append(node.record)

        # Inserção dos filhos na pilha para continuidade da varredura
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)

    return results
```

2. Análise de Complexidade Teórica

A rotina apresentada possui as seguintes características assintóticas:

- **Complexidade de Tempo:** $O(N)$. Como não há ordenação prévia baseada no campo secundário, o pior caso exige a visita e verificação de todos os N nós da árvore.

- **Complexidade de Espaço:** $O(H)$, onde H é a altura da árvore (espaço consumido pela pilha em memória). Em árvores balanceadas (AVL), $H = O(\log N)$. Em árvores BST degeneradas no pior caso, $H = O(N)$.

3. Comportamento Empírico e o Paradoxo do Paralelismo

Apesar da complexidade de tempo linear $O(N)$ sugerir lentidão teórica, os testes empíricos do ambiente (*Bare-Metal* com interrupção do *Garbage Collector*) demonstraram que a execução sequencial (*Single-Thread*) dessa rotina apresentou um desempenho extremamente satisfatório. O algoritmo varreu a massa máxima de ~49.000 nós em poucos milissegundos.

Isso ocorre porque a operação ocorre inteiramente na memória RAM, executando apenas comparações simples de referências, sem o gargalo de acessos a disco (I/O) ou latência de rede comuns em bancos de dados SQL tradicionais.

Ao tentar otimizar esta busca linear dividindo a varredura entre múltiplos núcleos (*Multiprocessing* em 4 Cores), o resultado foi uma regressão severa de desempenho. O custo computacional arquitetural para fatiar o trabalho — serializar os nós da árvore (*Pickling*) e trafegar esses dados via Comunicação Inter-Processos (IPC) — foi ordens de grandeza superior ao tempo que um único núcleo dedicado leva para iterar a memória localmente. Isso atesta que, para operações de varredura atômica em memória local, a via sequencial nativa em Python é superior à paralelização.

4. Considerações Finais de Implementação

A implementação de varredura $O(N)$ apresentada mostrou-se prática e eficiente para o escopo do projeto (~49 mil registros). Do ponto de vista da engenharia de software, esta arquitetura é recomendada apenas quando:

- As consultas por atributos secundários são esporádicas ou de caráter analítico (*ad-hoc*).
- O dataset repousa integralmente na memória volátil (RAM).

Para cenários produtivos onde requisições por campos secundários sejam frequentes e escalem para milhões de registros, essa implementação iterativa pode ser substituída por estruturas de Índices Auxiliares (*Hash Maps* $O(1)$), conforme discutido na fundamentação teórica prévia.

7. Conclusão

Este trabalho apresentou uma análise empírica e teórica rigorosa sobre o comportamento de Árvores Binárias de Busca (BST) e Árvores auto-balanceadas (AVL), contrastando a complexidade assintótica clássica com os desafios arquiteturais da linguagem Python e do hardware moderno.

Os experimentos evidenciaram que a eficiência de uma estrutura de dados não depende estritamente de seu limite matemático teórico, mas é fortemente influenciada pela entropia dos dados de entrada e pelo custo oculto do ambiente de execução. Enquanto a árvore AVL demonstrou robustez nos cenários avaliados — garantindo operações em $O(\log N)$ mediante o custo contínuo de rotações de balanceamento —, a árvore BST provou ser topologicamente volátil. Em cenários ordenados, a BST sofre degeneração estrutural crítica para $O(N)$; contudo, sob alta entropia (dados aleatórios), sua simplicidade algorítmica permite que ela supere a própria AVL ao dispensar o *overhead* de balanceamento.

Um dos achados mais relevantes deste experimento foi a demonstração prática da Lei de Amdahl aplicada à arquitetura de concorrência do Python. A tentativa de escalar horizontalmente as operações de busca via *Multiprocessing* resultou em degradação severa de desempenho nas estruturas eficientes. O pesado custo infraestrutural de serialização de objetos (*Pickling*) e de comunicação inter-processos (IPC) obscureceu completamente os ganhos computacionais. A paralelização provou-se útil apenas como medida atenuante no pior caso teórico da BST (estruturas degeneradas), confirmando a premissa de que a força bruta computacional não substitui o planejamento e a eficiência algorítmica.

Por fim, a investigação sobre pesquisas baseadas em campos secundários expôs a limitação unidimensional intrínseca das topologias binárias. Requisições fora da chave primária de indexação anulam as garantias direcionais da árvore, forçando a degradação para varreduras iterativas de complexidade linear $O(N)$. Para a evolução dessa arquitetura visando cenários produtivos de alta demanda, a literatura e os dados apontam para a adoção de abordagens híbridas, como o acoplamento de Índices Secundários (tabelas *Hash* de acesso $O(1)$) associados aos nós, unindo a velocidade matemática das árvores com a flexibilidade de acesso horizontal dos dicionários.

7. Referências

AMDAHL, Gene M. *Validity of the single processor approach to achieving large scale computing capabilities*. In: Proceedings of the Spring Joint Computer Conference. AFIPS, 1967. p. 483–485.

ARAVARAVIND. *WB Commodity Price Intelligence (1960–2026)*. Kaggle, [ano]. Disponível em: <https://www.kaggle.com/code/aravaravind/wb-commodity-price-intelligence-1960-2026>. Acesso em: 20 mar. 2026.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

GOODRICH, Michael T.; TAMASSIA, Roberto; GOLDWASSER, Michael H. *Data Structures and Algorithms in Python*. Hoboken: Wiley, 2013.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. *Sistema de banco de dados*. 6. ed. Rio de Janeiro: Elsevier, 2012.